

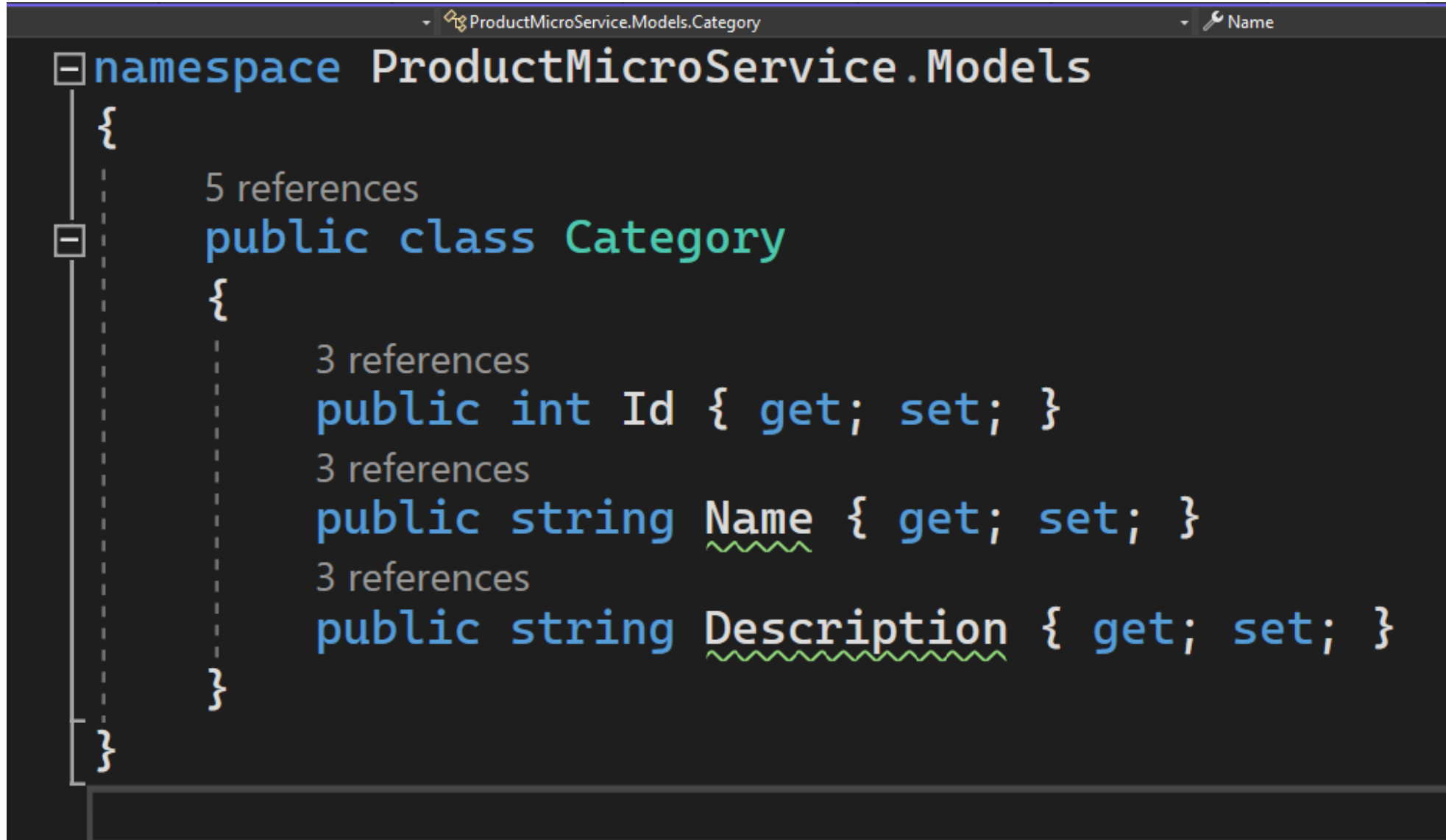
Developing Microservice in .NET Core

Create a new project of web API type

Models → Product.cs

```
ProductMicroService.Models.Product
namespace ProductMicroService.Models
{
    11 references
    public class Product
    {
        1 reference
        public int Id { get; set; }
        0 references
        public string Name { get; set; }
        0 references
        public string Description { get; set; }
        0 references
        public decimal Price { get; set; }
        0 references
        public int CategoryId { get; set; }
    }
}
```

Models → Category.cs



```
ProductMicroService.Models.Category Name
namespace ProductMicroService.Models
{
    5 references
    public class Category
    {
        3 references
        public int Id { get; set; }
        3 references
        public string Name { get; set; }
        3 references
        public string Description { get; set; }
    }
}
```

Models → ProductContext.cs

7 references

```
public class ProductContext : DbContext
```

```
{
```

0 references

```
public ProductContext(DbContextOptions<ProductContext> options) : base(options) { }
```

4 references

```
public DbSet<Product> Products { get; set; }
```

0 references

```
public DbSet<Category> Categories { get; set; }
```

0 references

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Category>().HasData(
new Category
{
    Id = 1,
    Name = "Electronics",
    Description = "Electronic Items",
},
new Category
{
    Id = 2,
    Name = "Clothes",
    Description = "Dresses",
},
new Category
{
    Id = 3,
    Name = "Grocery",
    Description = "Grocery Items",
}
```

Repository → IProductRepository

```
ctMicroService
ProductMicroService.Repository

1  using ProductMicroService.Models;
2
3  namespace ProductMicroService.Repository
4  {
5      4 references
6      public interface IProductRepository
7      {
8          2 references
9          IEnumerable<Product> GetProducts();
10
11          2 references
12          Product GetProductById(int ProductId);
13
14          2 references
15          void InsertProduct(Product product);
16
17          2 references
18          void UpdateProduct(Product product);
19
20          2 references
21          void DeleteProduct(int ProductId);
22
23          4 references
24          void Save();
25      }
26  }
```

Repository works as a micro component of microservice that encapsulates the data access layer and helps in data persistence and testability as well.

Repository → ProductRepository

```
ce
ProductMicroService.Repository.ProductRepository

using Microsoft.EntityFrameworkCore;
using ProductMicroService.Models;

namespace ProductMicroService.Repository
{
    2 references
    public class ProductRepository : IProductRepository
    {
        private readonly ProductContext _dbContext;

        0 references
        public ProductRepository(ProductContext dbContext)
        {
            _dbContext = dbContext;
        }

        2 references
        public void DeleteProduct(int ProductId)
        {
            var product = _dbContext.Products.Find(ProductId);
            _dbContext.Products.Remove(product);
            Save();
        }
    }
}
```


2 references

```
public Product GetProductById(int ProductId)
{
    return _dbContext.Products.Find(ProductId);
}
```

2 references

```
public IEnumerable<Product> GetProducts()
{
    return _dbContext.Products.ToList();
}
```

2 references

```
public void InsertProduct(Product product)
{
    _dbContext.Add(product);
    Save();
}
```

4 references

```
public void Save()  
{  
    _dbContext.SaveChanges();  
}
```

2 references

```
public void UpdateProduct(Product product)  
{  
    _dbContext.Entry(product).State = EntityState.Modified;  
    Save();  
}
```

Add Connection String

/json.schemastore.org/appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "ProductDB": "Data Source=(localdb)\\mssqllocaldb;Initial Catalog = ProductsDB;Integrated Security=True;"
  }
}
```

Add Configuration in Program.cs

```
20250121091552_init.cs  ProductsController.cs  ProductRepository.cs  IProductRepository.cs  appsettings.json  ProductContext.cs*  Category.cs  Program.cs*  Product.cs

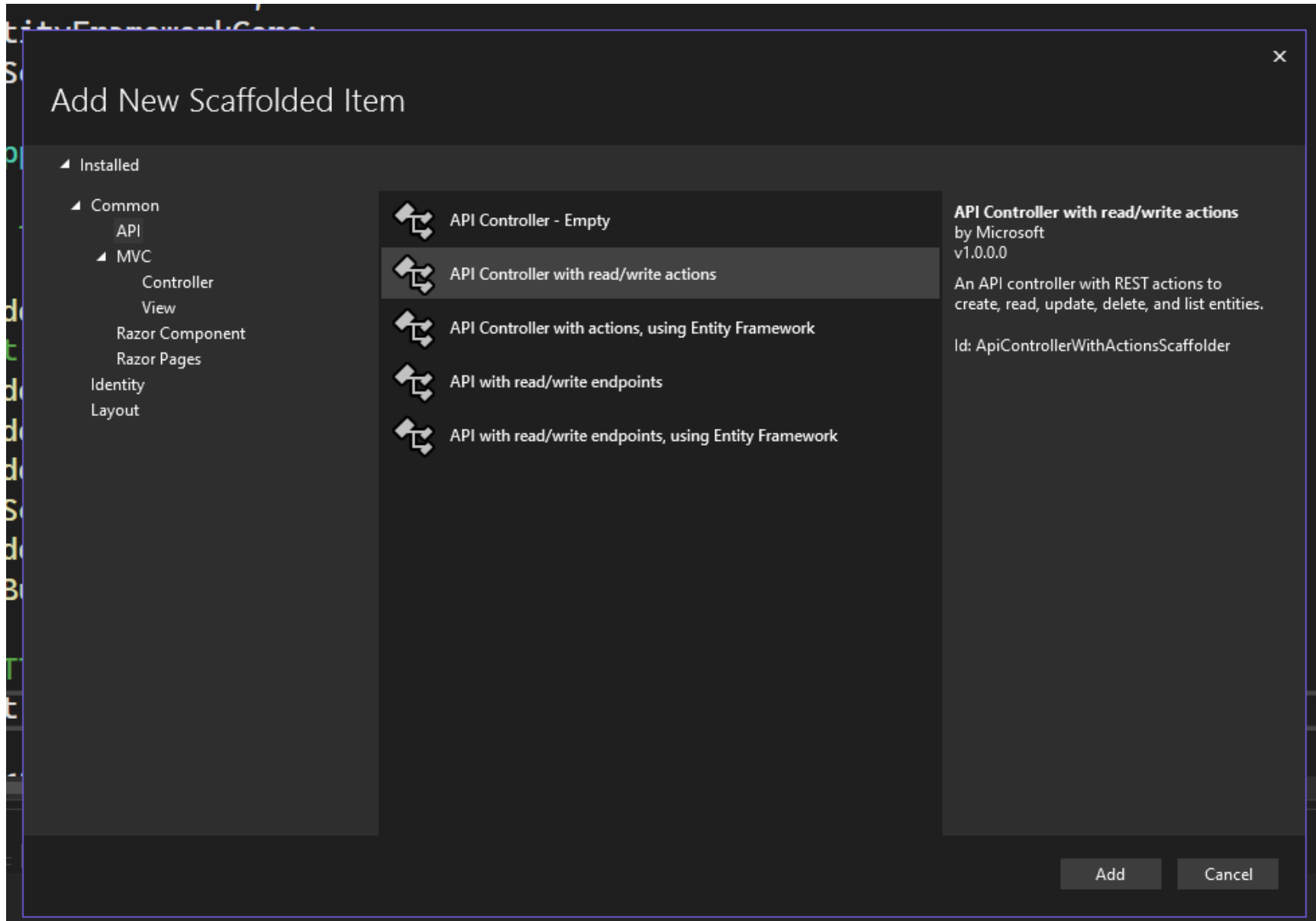
using Microsoft.Extensions.Configuration;
using ProductMicroService.Models;
using Microsoft.EntityFrameworkCore;
using ProductMicroService.Repository;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddDbContext<ProductContext>
    (o => o.UseSqlServer(builder.Configuration.GetConnectionString("ProductDB")));
builder.Services.AddTransient<IProductRepository, ProductRepository>();
var app = builder.Build();
```

Add Controller



ProductsController.cs

```
ProductMicroService.Controllers.ProductsController  Get(int id)

[Route("api/[controller]")]
[ApiController]
1 reference
public class ProductsController : ControllerBase
{
    private readonly IProductRepository _productRepository;

    0 references
    public ProductsController(IProductRepository productRepository)
    {
        _productRepository = productRepository;
    }
    // GET: api/<ProductsController>

    [HttpGet]
    1 reference
    public IActionResult Get()
    {
        var products = _productRepository.GetProducts();
        return new OkObjectResult(products);
    }
}
```

```
// GET api/<ProductsController>/5
```

```
[HttpGet("{id}", Name = "Get")]
```

1 reference

```
public IActionResult Get(int id)
```

```
{
```

```
    var product = _productRepository.GetProductById(id);
```

```
    return new OkObjectResult(product);
```

```
}
```

```
[HttpPost]
```

0 references

```
public IActionResult Post([FromBody] Product product)
```

```
{
```

```
    using (var scope = new TransactionScope())
```

```
    {
```

```
        _productRepository.InsertProduct(product);
```

```
        scope.Complete();
```

```
        return CreatedAtAction(nameof(Get), new { id = product.Id }, product);
```

```
    }
```

```
}
```

```
[HttpPut]
```

0 references

```
public IActionResult Put([FromBody] Product product)
{
    if (product != null)
    {
        using (var scope = new TransactionScope())
        {
            _productRepository.UpdateProduct(product);
            scope.Complete();
            return new OkResult();
        }
    }
    return new NoContentResult();
}
```

```
// DELETE api/<ProductsController>/5
```

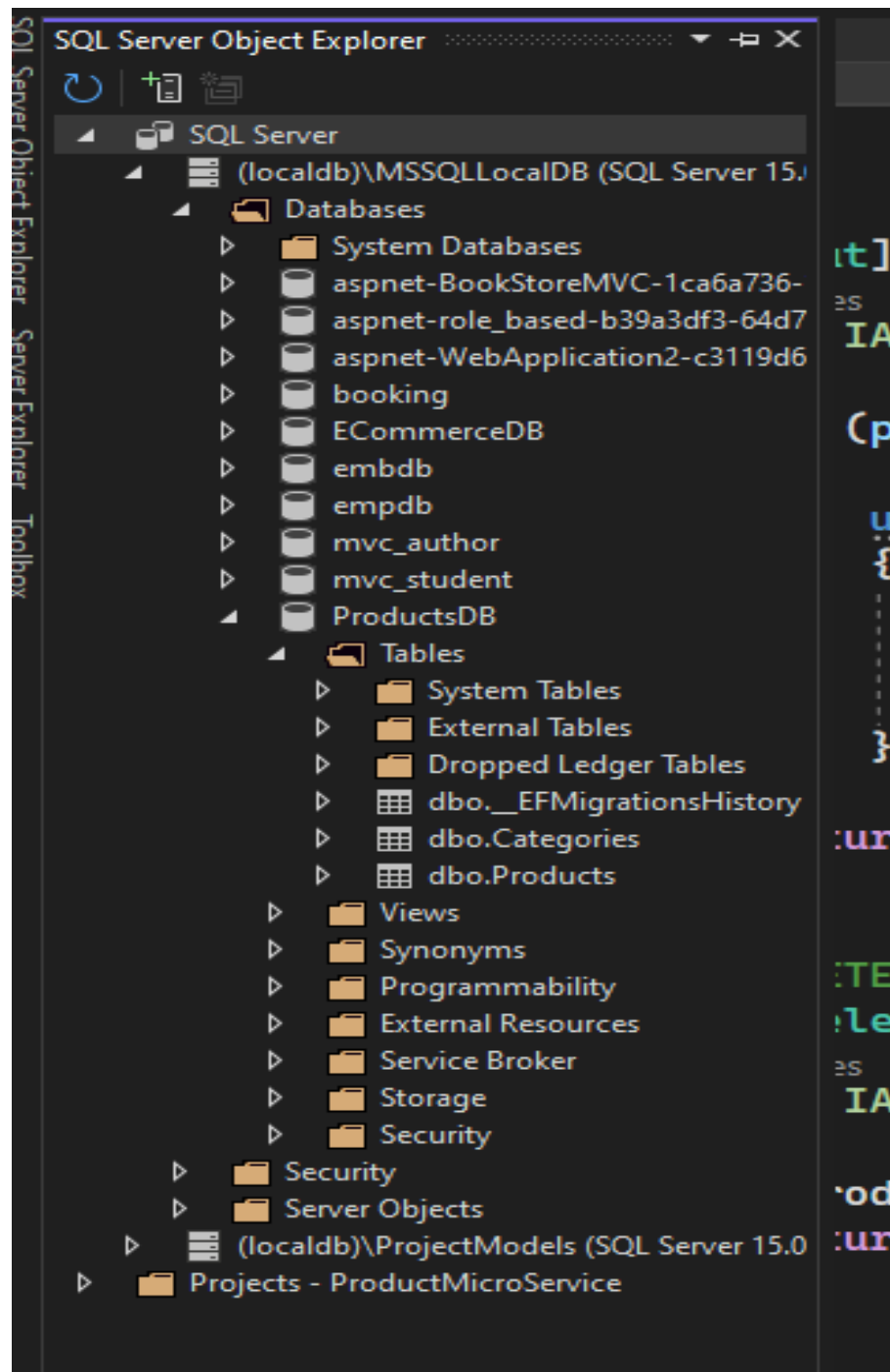
```
[HttpDelete("{id}")]
```

0 references

```
public IActionResult Delete(int id)
{
    _productRepository.DeleteProduct(id);
    return new OkResult();
}
```

```
}
```


Perform Migration
and verify the
database and
table creation



Test the endpoints

Standard Web API vs Microservice

Feature/Aspect	Microservice	Standard Web API
Definition	A small, independently deployable service designed around a specific business capability.	A traditional web API designed to expose endpoints for client-server communication.
Architecture	Follows a distributed architecture with multiple services.	Typically monolithic or layered architecture.
Scalability	Highly scalable; individual services can scale independently.	Limited scalability; the entire application scales as a single unit.
Development Complexity	Higher complexity due to distributed systems, inter-service communication, and deployment pipelines.	Lower complexity as everything is within a single application.
Deployment	Independent deployment of services; supports CI/CD pipelines.	Single deployment unit for the entire application.
Fault Isolation	High; issues in one service do not impact others.	Low; issues in one module can affect the entire application.
Technology Stack	Allows polyglot programming; different services can use different technologies.	Limited to a single technology stack.
Communication	Uses lightweight protocols like HTTP/REST, gRPC, or message brokers (e.g., RabbitMQ, Kafka).	Primarily uses HTTP/REST for communication.
Database Design	Database-per-service or shared databases (rarely); supports decentralization.	Typically uses a shared database.

Feature/Aspect	Microservice	Standard Web API
Team Structure	Supports cross-functional, service-aligned teams.	Centralized team structure.
Performance	Network latency between services can introduce overhead.	Lower latency as all components are tightly coupled.
Testing	More challenging due to distributed nature; requires tools for integration and contract testing.	Easier to test with a single codebase.
Monitoring and Logging	Requires distributed tracing and centralized logging tools like ELK, Prometheus, or OpenTelemetry.	Simplified monitoring and logging setup.
Use Case	Suitable for large, complex, and scalable applications.	Suitable for smaller or less complex applications.
Cost	Higher operational cost due to infrastructure and monitoring needs.	Lower operational cost.